Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

DESARROLLO DE APLICACIONES MÓVILES

Actividad Autónoma N.º 09

Diferencias por plataforma y evidencia en iOS y Android

Dato	Detalle	Dato	Detalle
Asignatura	Desarrollo de Aplicaciones Móviles	Ciclo	VI · Semestre 2026-2
Unidad	2. Conectividad, CRUD REST y persistencia multiplataforma	Sesión	9 (Unidad 2, sesión 3)
Dedicación	2 horas de trabajo autónomo	Entrega	Lunes 12 de octubre de 2026, 23:59
Proyecto	PharmaMobil sobre PharmaSoft	Puntaje	20 puntos

1. Descripción de la actividad

En la sesión práctica declaraste tu primer expect en commonMain y escribiste los dos actual correspondientes: el formato de moneda peruana y la capacidad de compartir un producto. Esa práctica demostró que el mecanismo funciona. Esta actividad te pide algo distinto: demostrar que entiendes por qué cada plataforma lo resuelve como lo resuelve.

Elaborarás un inventario de las capacidades nativas de PharmaMobil, un informe comparativo que explique las diferencias de implementación entre Android e iOS, una tercera capacidad nativa implementada por tu cuenta y la evidencia de que todo funciona tanto en el emulador de Android como en el simulador de iOS.

Por qué importa

El 90 % del código de PharmaMobil es común y el 10 % restante es donde se concentran los errores difíciles. Documentar ese 10 % ahora significa que cuando en la sesión 11 agregues persistencia local con SQLDelight, que también requiere drivers distintos por plataforma, ya tendrás el patrón resuelto y el criterio para decidir qué baja al source set específico.

1.1 Propósito

Documentar y justificar las diferencias de implementación entre Android e iOS en las capacidades nativas de PharmaMobil, evidenciando con ejecuciones reales que el código común produce el comportamiento esperado en ambas plataformas.

1.2 Ficha del sílabo

Esta actividad responde literalmente al enunciado que el sílabo asigna a la actividad autónoma de la sesión. La tabla permite verificar esa correspondencia.

Elemento del sílabo	Contenido
Asignatura	Desarrollo de Aplicaciones Móviles · EP Ingeniería de Sistemas · Ciclo VI · Plan 2024-1 · Semestre 2026-2 · 3 créditos.
Docente	Mg. Reyna Barreto Benjamin David.
Unidad y sesión	Unidad 2, sesión 3 (sesión 9 del curso), del 06-10-2026. Dedicación autónoma: 2 horas.
Contenido de la sesión	«Mecanismo expect/actual, capacidades nativas por plataforma e interoperabilidad Kotlin-Swift.»
Enunciado de esta actividad	«Documenta diferencias por plataforma y evidencia iOS/Android.»

1.3 Competencias, resultado de aprendizaje y producto

Elemento	Descripción
Competencia específica	Ingeniería de Software: gestiona y desarrolla software de manera eficiente y efectiva, basándose en estándares internacionales de calidad. Dimensión 1.1 Programación y desarrollo de soluciones de software.
Competencia general	Carácter y aprendizaje autónomo: firmeza de propósito, ejecución, dominio propio y mantener el esfuerzo. Dimensión 1.1.
Resultado de aprendizaje U2	Integra conectividad REST, autenticación, manejo de errores y persistencia local offline-first en una aplicación KMP.
Producto de la Unidad 2	Aplicación KMP con CRUD REST completo con Ktor, autenticación JWT y persistencia local offline-first.
Criterio del producto U2	Código específico de plataforma correctamente aislado · Equivalencia funcional entre Android e iOS.
Peso en la evaluación	Componente «evaluación de sesiones»: 20 % de la nota final. Nota mínima aprobatoria: 13.

2. Productos a entregar

Elabora un único documento en PDF titulado «S09_ActividadAutonoma_Apellidos.pdf» con los cuatro productos siguientes, y actualiza tu rama del repositorio con los commits correspondientes.

Producto 1. Inventario de capacidades nativas

Recorre tu proyecto y registra cada declaración expect que exista, dónde vive su contrapartida y qué API nativa usa en cada lado. Incluye las dos de la práctica y la que implementes en el producto 3.

Capacidad	Firma del expect	API usada en Android	API usada en iOS
Formato de moneda	fun formatearSoles(valor: Double): String	NumberFormat con Locale("es", "PE")	NSNumberFormatter con NSLocale("es_PE")
Compartir producto	interface Compartidor	Intent.ACTION_SEND	UIActivityViewController
Módulo de inyección	val platformModule: Module	module con androidContext()	module sin contexto
La tuya (producto 3)	—	—	—

Para cada fila agrega una línea que indique en qué archivo y en qué source set está cada actual. La ruta exacta forma parte de la evidencia.

Producto 2. Informe comparativo de diferencias

Este es el núcleo de la actividad. Redacta un análisis de entre 600 y 900 palabras que responda las cinco preguntas siguientes con referencia a tu propio código, no en abstracto.

N.º	Pregunta a responder	Qué se espera de tu respuesta
1	¿Por qué la API de formato de moneda difiere entre plataformas si ambas usan el mismo locale?	Explicar que la JVM y Foundation implementan el estándar de forma distinta, y mostrar el resultado literal que produce cada una.
2	¿Por qué CompartidorAndroid necesita un Context y CompartidorIos no?	Describir el modelo de componentes de Android frente al de UIKit y qué entrega Koin en cada caso.
3	¿Qué habría cambiado si en lugar de expect/actual hubieras usado una interfaz con inyección de dependencias?	Comparar ambas estrategias en términos de acoplamiento, pruebas y legibilidad, y justificar la elegida.
4	¿Qué ocurre si una de las dos implementaciones actual no existe?	Reportar el error de compilación exacto que obtuviste al provocarlo deliberadamente.
5	¿Qué capacidad del proyecto NO debería bajar al código específico de plataforma y por qué?	Identificar un caso concreto de PharmaMobil y argumentar el criterio de decisión.

La pregunta 4 exige ejecución real

Comenta temporalmente uno de los dos actual, intenta compilar y captura el mensaje del compilador. Luego descoméntalo. El mensaje exacto —no una paráfrasis— es parte de la evidencia.

Producto 3. Una tercera capacidad nativa, por tu cuenta

Elige una capacidad de la lista siguiente, declárala como expect en commonMain e impleméntala en ambos source sets. Debe quedar conectada a la interfaz de PharmaMobil, no como código suelto.

Opción	Qué debe hacer	Pista de la API nativa
Información del dispositivo	Mostrar sistema operativo y versión en una pantalla «Acerca de».	Build.VERSION.RELEASE · UIDevice.currentDevice.systemVersion
Apertura de enlaces	Abrir el enlace del laboratorio del producto en el navegador del sistema.	Intent.ACTION_VIEW · UIApplication.sharedApplication.openURL
Formato de fecha	Mostrar fechaCreacion del producto como «3 de octubre de 2026».	SimpleDateFormat · NSDateFormatter
Copiado al portapapeles	Copiar el código del producto al portapapeles con confirmación visible.	ClipboardManager · UIPasteboard.generalPasteboard

Si prefieres otra capacidad, consúltala antes por el aula virtual. El requisito es que tenga implementación genuinamente distinta en cada plataforma: si ambas resultan idénticas, la capacidad no justificaba un expect y el producto no acredita.

```
// commonMain/platform/InfoDispositivo.kt
expect class InfoDispositivo() {
    val sistema: String
    val version: String
}
```

Punto de control

Antes de dar por terminado el producto 3, busca en todo el paquete commonMain las importaciones android. y platform. No debe aparecer ninguna. Si aparece una, la capacidad quedó mal aislada y el proyecto dejará de compilar para iOS.

Producto 4. Evidencias de ejecución en ambas plataformas

- Captura del precio formateado en el emulador de Android y en el simulador de iOS, lado a lado, mostrando el mismo producto.
- Captura del selector de compartir de Android y de la hoja de compartir de iOS, ambas con el texto del producto visible.
- Captura de la tercera capacidad funcionando en las dos plataformas.
- Captura del mensaje del compilador obtenido en la pregunta 4 del informe.
- Enlace a tu rama y a los commits de esta actividad.

- README del proyecto actualizado con una sección «Código específico de plataforma» que liste las capacidades del inventario.

Si no tienes acceso a una Mac

La compilación de iOS requiere Xcode y, por tanto, macOS. Si trabajas en Windows o Linux, coordina con un integrante del equipo que disponga de Mac para obtener las capturas de iOS, e indícalo explícitamente en el documento junto al nombre de quien ejecutó. El código iOS debes escribirlo tú de todos modos: se evalúa en el repositorio.

3. Orientaciones para el desarrollo

1. Ejecuta antes de escribir. El informe comparativo se evalúa por la precisión de lo observado, no por la elegancia de la redacción.
2. Copia los resultados literales: si NumberFormat produce «S/ 24.50» y NSNumberFormatter produce «S/24.50», esa diferencia de un espacio es exactamente el hallazgo que se busca.
3. Mantén la firma del expect y la del actual idénticas, incluidos los nombres de los parámetros: el compilador los compara uno a uno.
4. Verifica que el paquete declarado en la línea package sea el mismo en los tres archivos; es la causa más frecuente del error «actual no encontrado».
5. Registra un commit por producto en tu rama, con mensajes que indiquen la capacidad implementada.

Uso de herramientas de apoyo

Puedes apoyarte en la documentación oficial y en herramientas de asistencia, pero las capturas, los mensajes del compilador y el código entregado deben provenir de la ejecución real de tu propio proyecto. Cualquier evidencia que no corresponda al repositorio entregado invalida la actividad.

4. Criterios de evaluación

La actividad se califica sobre veinte puntos con la siguiente rúbrica analítica.

Criterio	Peso	Logrado (100 %)	En proceso (60 %)	Inicial (30 %)
Inventario de capacidades	3 pts	Todas las capacidades registradas con firma, source set, archivo y API nativa de cada lado.	Inventario completo pero sin rutas de archivo o sin la API nativa.	Solo lista las capacidades por nombre.
Informe comparativo	7 pts	Las cinco preguntas respondidas con referencia al código propio y resultados observados.	Tres o cuatro preguntas respondidas, o respuestas correctas pero genéricas.	Menos de tres preguntas, o respuestas copiadas de la documentación.

Criterio	Peso	Logrado (100 %)	En proceso (60 %)	Inicial (30 %)
Tercera capacidad nativa	6 pts	expect declarado, dos actual distintos, conectada a la interfaz y funcionando en ambas plataformas.	Implementada en ambos lados pero sin conectar a la interfaz, o con una sola plataforma verificada.	Solo un actual, o código que no compila.
Evidencias de ejecución	2 pts	Capturas de Android e iOS para cada capacidad, más el mensaje del compilador.	Capturas de una sola plataforma o faltan las de la tercera capacidad.	Sin capturas o capturas que no corresponden al proyecto.
Aislamiento y repositorio	2 pts	Sin importaciones de plataforma en commonMain, commits en la rama propia y README actualizado.	Aislamiento correcto pero sin README o con commits fuera de la rama.	Código de plataforma filtrado a commonMain.

5. Lista de cotejo previa a la entrega

Verifica cada punto antes de subir el archivo al aula virtual.

N.º	Criterio observable	Cumple (Sí / No)
1	El documento está en PDF y nombrado como S09_ActividadAutonoma_Apellidos.pdf.	
2	Incluye portada con nombre completo, sesión, fecha y enlace a la rama del repositorio.	
3	El inventario registra cada expect con su archivo, su source set y la API nativa de cada lado.	
4	El informe responde las cinco preguntas citando mi propio código.	
5	La respuesta a la pregunta 4 incluye el mensaje literal del compilador.	
6	La tercera capacidad tiene un expect y dos actual con implementaciones distintas.	
7	La tercera capacidad se ve funcionando desde la interfaz, no solo en una prueba.	
8	No existe ninguna importación android. ni platform. dentro de commonMain.	
9	Hay capturas de Android y de iOS para cada una de las tres capacidades.	
10	El README incluye la sección «Código específico de plataforma».	
11	Los commits de esta actividad están en mi rama y son accesibles desde el enlace entregado.	

6. Entrega y plazos

Aspecto	Detalle
Medio de entrega	Aula virtual, tarea «Sesión 9 — Actividad autónoma».
Fecha límite	Lunes 12 de octubre de 2026, hasta las 23:59.
Formato	Un archivo PDF más el enlace a la rama en el comentario de la entrega.
Entrega fuera de plazo	Se califica sobre el 70 % del puntaje durante las 48 horas siguientes.
Retroalimentación	Al inicio de la sesión 10 (13 de octubre de 2026).

Continuidad

En la sesión 10 se incorpora la autenticación con JWT y el almacenamiento seguro del token, que en Android usa EncryptedSharedPreferences y en iOS el Keychain. Es decir, otra capacidad nativa con dos implementaciones distintas: llegarás con el patrón que estás documentando aquí ya resuelto.

7. Referencias de apoyo

7.1 Referencias básicas del sílabo

- JetBrains. (2025b). Kotlin Multiplatform documentation. https://kotlinlang.org/docs/multiplatform.html
- JetBrains. (2025b). Expected and actual declarations. https://kotlinlang.org/docs/multiplatform-expect-actual.html
- JetBrains. (2025b). Kotlin/Native platform libraries. https://kotlinlang.org/docs/native-platform-libs.html
- JetBrains. (2025b). Interoperability with Swift/Objective-C. https://kotlinlang.org/docs/native-objc-interop.html
- Hogan, J. (2024). Mastering Kotlin Multiplatform: Build Cross-Platform Apps for Android, iOS, Web, and Desktop. Independently Published. ISBN 979-8-3461-9721-8.
- Touchlab. (2025). Kotlin Multiplatform resources. https://touchlab.co/
- JetBrains. (2025a). Compose Multiplatform. https://www.jetbrains.com/compose-multiplatform/

7.2 Recursos del curso

- PharmaMobil, aplicación del curso. https://github.com/dreyna/pharmaMobil
- PharmaSoft, backend del curso. https://github.com/dreyna/pharmaSoft